

组件间交互规划

本交付包当前只包含五个组件自身功能：Tree、Chord、Prism、Scroll、List。本文件用于说明后续可以实现哪些组件间交互，以及这些交互如何按实际挂载组件动态启用。

目标

后续联动不应把五个组件写死成一个大页面，而应采用“调用哪几个组件，就启用哪几个组件之间的交互”的方式：

- 只调用一个组件：只保留组件内部 hover、click、highlight。
- 调用两个组件：只启用这两个组件之间的同步。
- 调用多个组件：根据已挂载组件集合自动组合交互。
- List 作为可选详情面板，接收任意组件点击结果。

example/ 中保留的源系统可作为交互效果参考。源系统中的核心链路包括：

- Tree 点击 slice 后进入当前 slice。
- Prism 当前可见面和下方 slice 标签同步当前 slice。
- Chord 根据当前 slice 高亮 arc/ribbon。
- Scroll 展示当前 slice 的点边图，点击节点或边后更新右侧详情。
- Scroll 左右 stream hover/click 时高亮对应上下文边。
- List/Paper List 行 hover 时反向高亮 Scroll 中的节点。

Topic Tree 说明

Topic Tree 在产业通系统中已经有较成熟的实现；同时 reference 原系统并没有复杂 Tree 组件，只是在左侧保留了 topic/slice 的列表入口。因此本交付包中的 Tree 只做了简单列表式层级展示，用于配合功能演示和触发 slice 选择，没有继续投入复杂树布局、展开收起动画、检索、拖拽等深入开发。

如果甲方需要完整的产业层级树，建议直接复用产业通已有实现，并通过统一事件接口与本包其他组件联动。

推荐事件

后续 coordinator 可以围绕以下事件实现：

```
slice: hover
slice: select
slice: clear
entity: hover
entity: select
entity: clear
relation: hover
relation: select
relation: clear
contextSlice: hover
contextSlice: select
contextSlice: clear
view: change
layout: change
detail: show
detail: clear
```

事件 payload 建议统一包含：

```
{
  type: "slice" | "entity" | "relation" | "contextSlice",
  id,
  sourceComponent: "tree" | "chord" | "prism" | "scroll" | "list",
  item,
  context,
}
```

两两交互清单

组件组合	原系统已有逻辑	后续实现说明
Tree -> Prism	Tree 点击 slice 后，源系统通过 slice 选择逻辑进入对应 topic；Prism 可旋转到对应面并同步当前 slice 状态。	Tree 触发 <code>slice:select</code> ；如果 Prism 已挂载，则高亮/旋转到对应 face，并同步下方 slice tag。
Prism -> Tree	源系统中 Prism 旋转导致当前可见 face 变化时，会同步左侧 Tree 的 active 状态。	Prism 触发 <code>slice:hover</code> 或 <code>slice:select</code> ；Tree 只做 active 行高亮，不强制滚动，避免干扰用户阅读。
Tree -> Chord	Tree 选择 slice 后，当前 slice 状态会传递给 Chord，使对应 arc 和相关 ribbon 高亮。	Tree 触发 <code>slice:select</code> ；如果 Chord 已挂载，调用 Chord 高亮指定 slice。
Chord -> Tree	源系统中 Chord hover 主要做 Chord 内部高亮和 tooltip，Tree 反向同步较弱。	建议新增：Chord arc 点击时触发 <code>slice:select</code> ，Tree 高亮对应行；Chord hover 只临时高亮，不改变 Tree selection。
Tree -> Scroll	Tree 点击可选择 slice 后，源系统会打开或切换到该 slice 的 Scroll/GeneticFlow。	Tree 触发 <code>slice:select</code> ；如果 Scroll 已挂载，更新 Scroll 的 <code>data</code> 和 <code>sliceId</code> 。
Scroll -> Tree	源系统中 Scroll 当前 slice 与全局 selected slice 同步，Tree 可显示 active 行。	Scroll 内部切换 slice 或外部 slice tag 切换时，Tree 同步 active 行。节点/边选择不影响 Tree。
Tree -> List	原系统没有把 Tree 点击直接展示为详情表。	建议新增：Tree 点击 slice 时，如果 List 已挂载，展示 slice 名称、节点数、关系数、颜色、层级等 slice 摘要。
List -> Tree	原系统没有 List 反向控制 Tree 的核心逻辑。	可选新增：List 当前展示 slice 时，点击标题或定位按钮可让 Tree 高亮并滚动到该 slice。
Chord -> Prism	源系统中 Prism 当前面变化会驱动 Chord；Chord 自身 hover 会高亮 arc/ribbon。	建议补齐反向：Chord arc 点击触发 <code>slice:select</code> ，Prism 旋转/高亮对应 face；Chord ribbon 点击可高亮 source/target 两个 face。
Prism -> Chord	源系统已有：Prism 旋转更新 active face，Chord 调用 arc/ribbon 高亮逻辑同步当前 slice。	Prism 触发 <code>slice:hover</code> 或 <code>slice:select</code> ；Chord 高亮对应 arc 及其出入 ribbon。
Chord -> Scroll	原系统中全局 slice 选择可以驱动 Scroll，但 Chord ribbon 到 Scroll context 的语义没有完全产品化。	Chord arc 点击打开对应 slice 的 Scroll；Chord ribbon 点击时，如果 Scroll 已挂载，则切换到 source slice 并高亮 target context，或按配置切换到 target slice。
Scroll -> Chord	源系统中 Scroll 当前 slice 与全局 active slice 一致，Chord 可同步高亮当前 slice。	Scroll 切换 slice 时触发 <code>slice:select</code> ；Chord 高亮当前 slice。Scroll stream hover 可触发 <code>contextSlice:hover</code> ，Chord 临时高亮当前 slice 与 context slice 的 ribbon。
Chord -> List	原系统 Chord hover 主要展示 tooltip，没有稳定的详情表。	建议新增：Chord arc 点击展示 slice 摘要；Chord ribbon 点击展示 source slice、target slice、关系数、方向和占比。
List -> Chord	原系统没有明显的 List 反向控制 Chord。	可选新增：List 展示 slice 或 slice relation 时，反向高亮 Chord 中对应 arc/ribbon。
Prism -> Scroll	源系统已有：点击 Prism face 或 slice tag 后进入当前 slice 的 Scroll；再次选择当前 slice 可以返回/关闭 Scroll。	Prism 触发 <code>slice:select</code> ；Scroll 更新到对应 slice。支持 <code>view:change</code> 在 Prism 和 Scroll 之间切换。
Scroll -> Prism	源系统中返回 Prism 后，当前 slice 可以继续保持高亮。	Scroll 的返回、关闭或 slice 切换触发 <code>slice:select</code> 或 <code>view:change</code> ；Prism 同步高亮/旋转。
Prism -> List	源系统中 Prism/Scroll 切换时右侧 Paper List 会更新为当前 slice 的节点列表。	Prism 选择 slice 后，如果 List 已挂载，展示该 slice 下节点列表或 slice 摘要。
List -> Prism	原系统 Paper List 行 hover 主要高亮 Scroll 节点；对 Prism face 的反向控制较弱。	可选新增：List 展示 slice 时高亮 Prism face；List 展示 entity 时高亮 entity 所属 slice face。
Scroll -> List	原系统已有：Scroll 节点点击展示 Paper/Entity Info；边点击展示 Citation/Relation Info；stream/context 点击可展示上下文信息。	Scroll 触发 <code>entity:select</code> 、 <code>relation:select</code> 、 <code>contextSlice:select</code> ；List 根据 <code>displayInfo</code> 展示语义清洗后的详情。
List -> Scroll	原系统已有：Paper List 行 hover 会调用节点高亮逻辑，使 Scroll 中相关节点和邻边突出。	List 行 hover 触发 <code>entity:hover</code> ；Scroll 高亮节点、邻居和相关边。List 中 relation 行 hover 可高亮对应边。

原系统关键交互逻辑映射

原系统函数/行为	交互含义	后续组件化接口
<code>bindTreePanel()</code>	左侧 topic/slice 列表点击。	Tree <code>onSelect(slice)</code> -> <code>slice:select</code>
<code>handleSliceTagClick()</code>	点击 slice tag/tree/prism 后切换当前 slice 或 Scroll 视图。	<code>selectSlice(sliceId, source)</code>

原系统函数/行为	交互含义	后续组件化接口
<code>updateOpacity()</code>	Prism 旋转时判断当前可见面并同步 active slice。	Prism <code>onActiveSliceChange(sliceId)</code>
<code>syncChordSelectionBySliceId()</code> / <code>highlight_arc_with_cash()</code>	Chord 根据当前 slice 高亮 arc/ribbon。	Chord <code>highlightSlice(sliceId)</code>
<code>highlightSlice()</code>	高亮某个 slice 的节点、上下文区域、tag 和 Chord/Scroll 相关元素。	coordinator 派发 <code>slice: hover/select</code>
<code>highlight_node()</code>	高亮 Scroll 节点、邻居、相关边，并显示节点详情。	Scroll <code>highlightEntity(entityId)</code> + List <code>showEntity</code>
<code>clickEdge()</code> / <code>showEdgeInfo()</code>	高亮 Scroll 边并显示关系详情。	Scroll <code>highlightRelation(relationId)</code> + List <code>showRelation</code>
<code>mouseoverStreamSegment()</code>	左右 stream hover 时高亮 stream 段和对应 context edges。	Scroll <code>contextSlice: hover</code>
<code>drawStreamgraph()</code> 中 stream click	固定某个 stream context 并展示相关点/边。	Scroll <code>contextSlice: select</code>
Paper List 行 <code>onmouseover</code>	列表行 hover 反向高亮 Scroll 节点。	List <code>onHover(entity)</code> -> Scroll <code>highlightEntity</code>

新增交互建议

以下交互不是原系统稳定显式实现，但从组件化联调角度建议补上：

- Chord arc 点击 -> Prism/Scroll/List 同步当前 slice。
- Chord ribbon 点击 -> Scroll 高亮 source-target 的上下文关系，List 展示聚合关系摘要。
- Tree 点击 -> List 展示 slice 摘要。
- List 展示 relation 时 hover -> Scroll 高亮对应关系边。
- List 展示 slice relation 时 hover -> Chord 高亮对应 ribbon。
- 所有 hover 类联动都应是临时状态，mouseleave 后恢复；所有 click 类联动才进入持久 selected 状态。

实现方式

建议新增独立 interaction coordinator，但不要让组件互相直接 import：

```
const coordinator = createInteractionCoordinator();

coordinator.register("tree", treeInstance);
coordinator.register("chord", chordInstance);
coordinator.register("prism", prismInstance);
coordinator.register("scroll", scrollInstance);
coordinator.register("list", listInstance);
```

coordinator 内部根据已注册组件决定是否执行联动：

```
coordinator.emit("slice:select", {
  id: sliceId,
  sourceComponent: "tree",
  item: slice,
});
```

如果只注册 Tree + Scroll，就只执行 Tree/Scroll 交互；如果注册 Prism + Chord + List，就只执行这三者之间的交互。未注册组件对应动作直接跳过，不报错、不创建隐藏依赖。

验收标准

- 单组件 demo 不受 coordinator 影响。
- 任意两个组件组合时，对应两两交互可用。

- 多组件组合时，事件不会循环触发或重复渲染。
- hover 状态和 selected 状态分离，mouseleave 不清除 click selection。
- List 只显示语义清洗后的字段，不直接展开全部 raw/meta。
- Scroll、Prism、Chord 的高亮颜色和选中状态与 `example/` 的视觉效果保持一致。